1 Introduction

This evidence looks at finding an automaton to accept a provided language. The resulting automaton can than be either modeled using Prolog or REGEX. The language chosen for this evidence posesses the alphabet $\Sigma = \{0, 1, 2\}$, needs to end in either "111" or "122" to transition into an accepting state and prohibits the occurence of "211".

2 Automaton

As [**?**] mention, (deterministic) finite automaton are useful models for different tasks, including lexical analysis. When interpreting finite automatons as transition graphs, this can be easily visulaized. A string can be broken down into each letter, which can be used to follow a graph into a specific node/ state. In [**?**] a deterministic finite automaton is defined as a quintuple $A = (Q, \Sigma, \delta, q0, F)$ where

- Q defines a finite set of states,
- Σ a finite alphabet,
- δ a transition function,
- q0 the inital state and
- F the accepting states.

An input gets accepted if, after passing the string, it ends up in an accepting state, otherwise it is rejected. The alphabet Σ is the same as mentioned in **??** and the other elements are yet to be determined.

While [**?**] also mention the existence of non-deterministic finite automaton, due to the small size and lower complexity of the language, a deterministic model approach was chosen. As [**?**] explain, a non-deterministic model has to be converted to a deterministic model, in order to be implementable. As using a deterministic model was possible, this option was more reasonable.

Each string should start out at the same entry state E. From there one can derive the different conitions path, the automaton needs to recognize. In **??** the different paths are depicted. The path s1e-s1f shows the accepted ending sequence "111", s1e-s2f models the accepting end string "122" and s3e-s3f show the rejected sequence "211". These are also all the required states, as any other sequence has no influence on acceptance or reejction and can lead back to E. With this F is defined as $F = \{s1f, s2f\}$. In **??**, the full model is shown with all connections. Important to note is, that each node needs to model each possible alphabet transition. Interesting observations include, that each 0 loops back to node E as 0 is no part of any important sequence, that once s3f is reached, all further transitions loop back to itself. Once this node has been reached, the string cannot be accepted. Further interesting to observer are the transitions

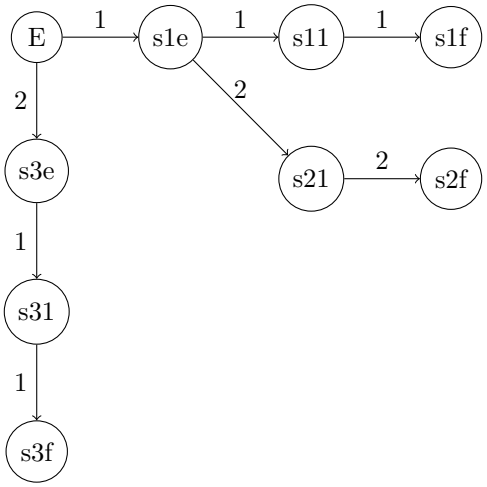

Figure 1: Base

between different branches, as each subsequence in a branch can potentially be the starting point of a different sequence.

3 Prolog

One method to implement the model

4 REGEX

$\hat{}$(0—1—20—22—212—210)*(111—122)$

5 Tests

5.1 Accepted

- 111
- 122
- 1122
- 00120122
- 111111122
- 01201201111

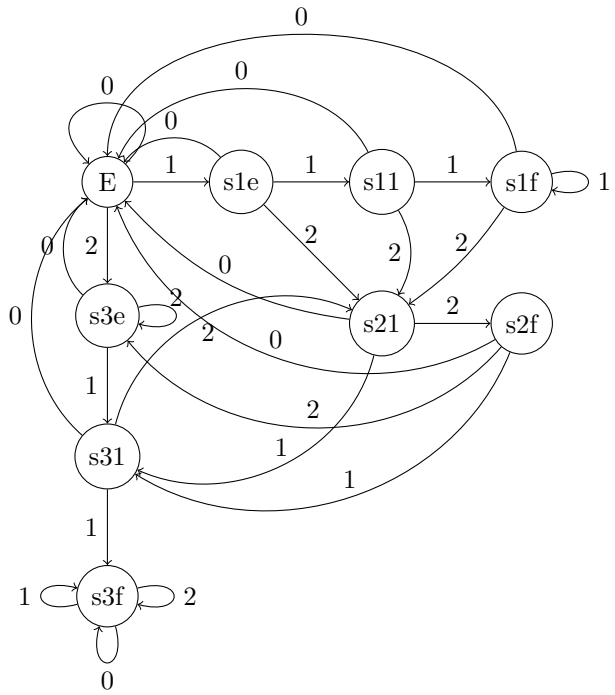

Figure 2: Base

5.2 Rejected

- 1112
- 211
- 01201202111
- 00121222
- 001211222
- 11112110111
- 2110102001221200122

References

[Hopcroft et al.2007] Hopcroft, J. E., Motwani, R., and Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation, 3rd edition.* Pearson Education, Inc.